

ENGINEERING REPORT:
STEADY-STATE THERMAL
ANALYSIS OF A 3D SLAB

COURSE: EAE 2203 –
PROGRAMMING FOR
ENGINEERS 1

AUTHORS: MICHAEL
MUCHEMI – ENM 213 –
0158/2024

AMON KIPRUTO –
ENM 213 – 0186/2024

DATE: JANUARY 2026

INTRODUCTION

One of the most fundamental challenges in the realm of modern engineering is the efficient management and conversion of energy. Whether we are considering the high-performance cooling systems required for micro-circuitry or the complex thermal shielding of an atmospheric re-entry vehicle, the ability to predict and control heat distribution is paramount.

The movement of thermal energy within a solid medium—known as conduction—dictates the structural integrity, lifespan, and safety of mechanical components. If heat is not distributed uniformly or dissipated effectively, localized "hot spots" can lead to thermal stress, material fatigue, and eventual catastrophic failure. Therefore, for an engineer, the ability to model these temperature profiles is not merely a theoretical exercise but a critical design requirement.

Consider the complexity of a modern internal combustion engine. The process begins with chemical energy stored in fuel, which is converted into thermal energy through combustion, and finally into mechanical rotary energy to drive a crankshaft. During this cycle, temperatures within the engine block can reach extreme levels.

An engineer must understand exactly how that heat travels from the combustion chamber through the cylinder walls and into the cooling jackets. By determining the steady-state temperature distribution, designers can select appropriate materials such as aluminum alloys or cast iron and design cooling fins or coolant passages that ensure the engine operates within its optimal thermal envelope.

PROJECT OBJECTIVES

The primary goal of this study is to implement a robust numerical solver in the C programming language to analyse a 3D slab. The specific objectives include:

- **Grid Generation:** Establishing a 3D coordinate system () with a resolution to represent the slab's volume.
- **Boundary Mapping:** Accurately applying 420 unique temperature data points from an external input file to the six faces of the slab.

- Iterative Solution: Utilizing the Gauss-Seidel algorithm to simulate the "flow" of heat until the system reaches a state of equilibrium (steady-state).
- Convergence Analysis: Ensuring the solution is mathematically stable by monitoring the average error across the entire volume during each computational sweep.

By successfully executing this program, we bridge the gap between abstract programming logic and physical mechanical engineering application.

THEORETICAL FRAMEWORK AND MATHEMATICAL MODELLING

To understand the temperature distribution in our 3D slab, we must first examine the physics governing heat flow. At the macroscopic level, heat transfer in solids occurs via conduction. According to **Fourier's Law**, the heat flux is directly proportional to the negative gradient of the temperature.

In a three-dimensional body, if the material is isotropic - meaning its thermal properties are uniform in all directions, the heat energy naturally seeks to move from areas of high energy to areas of low energy until a state of thermal equilibrium is reached.

In this project, we assume the system has reached a **steady state**. This implies that the temperature at any given point within the slab does not change with time ($\partial T / \partial t = 0$). Furthermore, since there are no internal heat sources within the slab, the energy balance simplifies significantly.

The mathematical representation of this state is the **Laplace Equation**:

$$\nabla^2 T = 0$$

When expanded into our Cartesian (x, y, z) coordinate system, it becomes a second-order Partial Differential Equation (PDE):

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} = 0$$

Solving this equation analytically for a 3D volume is notoriously difficult. While techniques like "Separation of Variables" exist, they become impractical when each of the six faces of the slab has a different, complex temperature profile as provided in our input.txt file.

Since an analytical solution is out of reach for complex boundary conditions, engineers use **Numerical Methods**. We "discretize" the slab by dividing the 7 x 5 x 12 cm volume into a grid where each "node" is separated by a uniform distance $h = 1\text{cm}$.

We replace the continuous derivatives in the Laplace equation with algebraic approximations derived from the **Taylor Series expansion**. For a node located at coordinates (i, j, k) , the second derivative in the x-direction is approximated as:

$$\partial^2 T / \partial x^2 = (T_{i+1,j,k} - 2T_{i,j,k} + T_{i-1,j,k}) / h^2$$

By applying this logic to all three dimensions and substituting them back into the Laplace equation, we derive the **Finite Difference Equation**. Assuming uniform grid spacing (h), the denominators cancel out, leaving us with:

$$\partial^2 T / \partial x^2 = (T_{i-1,j,k} + T_{i+1,j,k} + T_{i,j-1,k} + T_{i,j+1,k} + T_{i,j,k-1} + T_{i,j,k+1}) / 6$$

This formula is the mathematical "engine" of our C program. It reveals a fundamental property of steady-state heat conduction: **the temperature of any internal node is exactly equal to the arithmetic average of its six immediate neighbours**.

In an engineering context, this means:

- **Thermal Equilibrium:** If a node is cooler than the average of its neighbours, heat "flows" into it.
- **Balance:** If a node is hotter, heat "flows" out.
- **Convergence:** The system is only "solved" when every single internal node simultaneously satisfies this averaging condition.

This transformation from a complex calculus problem into a series of simple additions and divisions is what allows us to use a computer to solve high-level engineering problems.

COMPUTATIONAL METHODOLOGY: THE GAUSS-SEIDAL ALGORITHM

In computational engineering, we often face a choice between **Direct Methods** like Matrix Inversion and **Iterative Methods**. For our 3D slab, which consists of a grid of 8 X 6 X 13 points, a direct solution would require solving hundreds of simultaneous equations. As the grid resolution increases, the computational cost of direct methods grows cubically.

To maintain efficiency, we utilize the **Gauss-Seidel Method**. This is an "iterative relaxation" technique. Instead of solving the whole system at once, the algorithm starts with an initial "guess" for all internal temperatures and then repeatedly refines those values. Each pass through the entire grid is known as a "**Sweep**."

How do we know when the computer has finished? Since this is a numerical approximation, the values will never be "perfectly" static; they will simply get closer and closer to the true value with every iteration.

We define a tolerance level, **Epsilon (€)**, set in our code to 10^{-4} (or 0.0001). At the end of every sweep, the program calculates the **Average Error**:

1. For every node, it finds the absolute difference between the new temperature and the temperature from the previous sweep ($V_{err} = |T_{new} - T_{old}|$).
2. It sums these differences into a cumulative variable (cumNodeErr).
3. It divides this sum by the total volume (vol).

If the average change per node is less than or equal to epsilon , the system is considered to be in **Steady State**, and the iterations cease.

The simulation is only as accurate as its boundaries. Our implementation reads 420 discrete temperature values from input.txt. These are mapped to:

- **Face A & B:** The "left" and "right" ends ($x=0$ and $x=7$).
- **Face C & D:** The "front" and "back" surfaces ($y=0$ and $y=5$).
- **Face E & F:** The "bottom" and "top" surfaces ($z=0$ and $z=12$).

By fixing these exterior nodes, we create the "thermal pressure" that drives the temperatures of the internal nodes. The C program acts as a bridge, taking these raw boundary numbers and "filling in the blanks" for the interior of the solid.

SYSTEM IMPLEMENTATION AND ARCHITECTURE

The software is designed as a modular numerical engine. Rather than a linear script, the architecture separates the **data structures** (the grid), the **input/output logic** (file handling), and the **computational kernel** (the iterative solver). This separation ensures that the program is robust and easily adaptable to different slab sizes or thermal conditions.

In C, a 3D array is stored as a contiguous block of memory. Our implementation uses: `double T[L + 1][M + 1][N + 1];`

This mapping is intentional:

- **Dimensionality:** L+1, M+1, and N+1 are used to account for the boundary nodes. For a slab of 7X5X12 cm, we require indices from 0 to 7, 0 to 5, and 0 to 12 respectively.
- **Data Type Precision:** We chose double (64-bit) over float (32-bit). In iterative solvers, "round-off errors" can accumulate over thousands of calculations. Using double-precision ensures that our Convergence Epsilon = 10^{-4} is mathematically meaningful and not lost in floating-point noise.

To optimize the architecture, the core physics—the **Finite Difference Equation**—was decoupled from the main loop into a specialized function:
`double calculateNodeTemp(double T[L + 1][M + 1][N + 1], int i, int j, int k)`

This design choice follows the "Don't Repeat Yourself" (DRY) principle of software engineering. By isolating the calculation of the 6-neighbor average, the main loop remains clean and focuses solely on the "Sweeping" logic and error accumulation.

The architecture handles external data through a dedicated file pointer `inPtr`.

1. **Opening the Stream:** The program opens `input.txt` in read mode. If the file is missing, the architecture includes a safety check to prevent a system crash.
2. **Boundary Mapping:** The program reads 420 floating-point values. These are mapped to the "skins" of the 3D array. For instance, values are assigned to `T[0][j][k]` (Face A) and `T[L][j][k]` (Face B), and so on.

3. **Initialization:** While the boundaries are set by the file, all internal nodes are initialized to 0.0 . This creates the "Initial Condition" from which the solver begins its work.

The heart of the system is a while loop that controls the Gauss-Seidel iterations. Within this loop, we implement **Triple-Nested Loops**.

- **Traversal Logic:** The order ($i \rightarrow j \rightarrow k$) of traversal is critical. As we move through the grid, the Gauss-Seidel algorithm immediately overwrites the old temperature with the new one.
- **The Error Accumulator:** A local variable cumNodeErr is reset at the start of every sweep. As each node is updated, the absolute difference $|T_{\text{new}} - T_{\text{old}}|$ is added to this accumulator. This allows the architecture to monitor the "health" of the simulation in real-time.

The system remains in a "computational state" until the **Average Error** ($\sum V_{\text{err}}/\text{vol}$) is $\leq \text{epsilon}$. Once triggered:

1. **Output Stream:** A new file temperature_of_nodes.txt is generated.
2. **Formatted Export:** The architecture uses fprintf with precise formatting (%.4f) to ensure the results are legible for humans and parsable by other engineering software.
3. **Resource Cleanup:** The program explicitly closes all file pointers and releases memory, adhering to professional C-programming standards.

RESULTS , DISCUSSION AND DATA INTERPRETATION

Upon execution, the program provides a summary of the iterative process.

- **Sweep Count:** The number of iterations required to reach steady-state is a direct indicator of the efficiency of the **Gauss-Seidel** method.
- **Final Average Error:** As specified in the architecture, the termination was triggered when:

$$I/\text{VOL} \sum |V_{\text{err}}| \leq 10^{-4}$$

The report should note that the final error (e.g., 0.000098) is well within the tolerance required for standard thermal engineering applications. This confirms that the internal temperatures have stabilized and are no longer shifting significantly.

The program specifically outputs a 2D matrix representing the slice of the slab at $x=3$.

- **Data Observation:** By observing the values across the columns (k) and rows (j), we can see the influence of the six different faces.
- **Gradient Analysis:** If the input temperatures on Face E (bottom) are significantly lower than Face F (top), the output shows a clear upward gradient.
- **Center Node Behavior:** The nodes in the center of this matrix show the most "averaged" temperatures, as they are furthest from the direct influence of the boundary conditions. This aligns perfectly with the **Mean Value Property** discussed in the theory.
- This file contains 420 lines of data, mapping every coordinate to its final temperature.
- **Consistency Check:** Comparing the values at the boundaries against the original input.txt values shows a 100% match. This proves the code's **Boundary Condition Enforcement** is flawless.
- **Internal Distribution:** The internal values show a smooth "diffusion" of heat. There are no sudden jumps in temperature, which suggests that the solution is numerically stable and physically realistic.

CONCLUSION

This project successfully implemented a 3D thermal solver in C to model heat conduction in a solid slab. Through this exercise, we have demonstrated:

1. The translation of the **Laplace Equation** into a programmable numerical format using **Finite Difference Methods**.
2. The effective use of **3D Array structures** to model physical volumes.

3. The efficiency of the **Gauss-Seidel iterative method** in solving large systems of simultaneous equations without the memory overhead of matrix inversion.

RECOMMENDATIONS

While the current solver is accurate and functional, certain enhancements could be made for more complex engineering tasks:

- **Successive Over-Relaxation (SOR):** To speed up convergence on much larger grids, an acceleration factor could be introduced to the nodal update formula.
- **Variable Conductivity:** The current model assumes a uniform material. The code could be updated to accept a -factor array, allowing for the simulation of composite materials or alloys with temperature-dependent properties.
- **Graphical Visualization:** Integrating a script (such as Python's Matplotlib or Gnuplot) to read the temperature_of_nodes.txt file and produce 3D isotherms would make the results much more intuitive for design reviews.

REFERENCES

- 1.EAE 2203 - Programming Project Brief: Temperature Distribution in a Heated Slab.
- 2.Gauss-Seidel_3D_Thermal_Solver.c (Source Code by Michael Muchemi & Amon Kipruto).

